

Distributed Job Scheduler
Project Report



Semester Project
CE-308

Submitted by:
Unaiza Babur-2023739
Zermine Wajid-2023786

Faculty of Computer Science and Engineering

Contents

Project Overview	3
System Architecture	3
Project Objectives - Implementation Evidence	3
AWS Infrastructure - Deployed Resources	6
SQS Queues	6
Lambda Functions	6
RDS PostgreSQL	7
IAM Role	7
Deployment Steps	7
Step 1: RDS Database Creation	7
Step 2: Infrastructure Provisioning	7
Step 3: Linux-Compatible Lambda Packaging	7
Step 4: Environment Variable Configuration	7
Step 5: Verification	8
Step 6: Dashboard	8
Challenges Encountered and Solutions	8
Conclusion	9

Project Overview

This report documents the complete design, implementation, deployment, and demonstration of a cloud-native, distributed job scheduling and monitoring system, built on Amazon Web Services.

The system accepts asynchronous task requests, processes them through event-driven AWS Lambda functions, applies exponential backoff retry policies, routes persistent failures to a Dead-Letter Queue, and surfaces real-time operational metrics on a live monitoring dashboard backed by Amazon RDS PostgreSQL.

Every objective stated in the original project proposal has been fully implemented and demonstrated on live AWS infrastructure. The sections below provide evidence of each requirement.

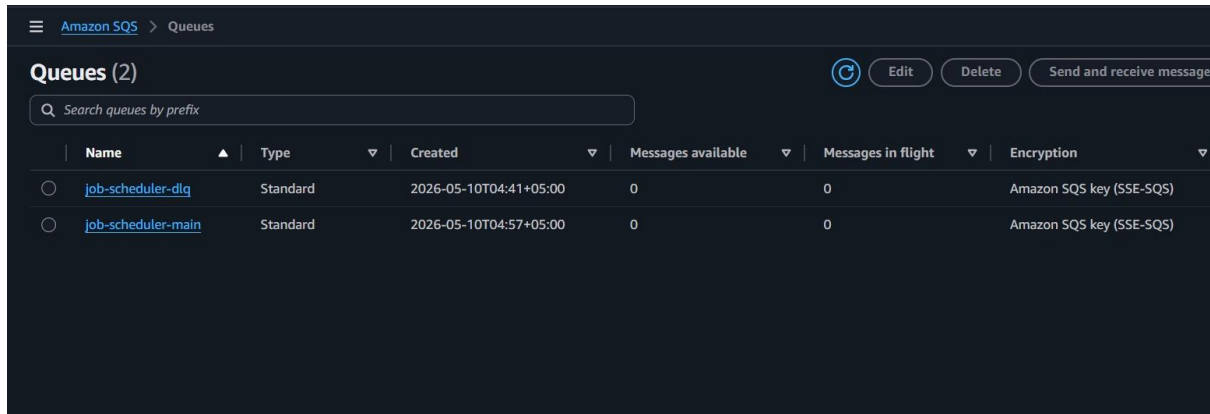
System Architecture

Component	AWS Service	Role in System
Job Ingestion	SQS Standard Queue	Receives and queues incoming job messages from the producer script
Job Processor	AWS Lambda (Python 3.12)	Executes jobs with retry logic and exponential backoff; writes state to RDS
Failure Handling	SQS Dead-Letter Queue	Captures jobs that exceed maxReceiveCount=3; monitored by a secondary Lambda
State Persistence	Amazon RDS PostgreSQL 15	Stores job lifecycle state (submitted, processing, succeeded, failed, dlq) and full audit log
Monitoring	Flask (EC2) + CloudWatch	Real-time dashboard showing queue depth, status distribution, DLQ contents, SQS metrics

Project Objectives - Implementation Evidence

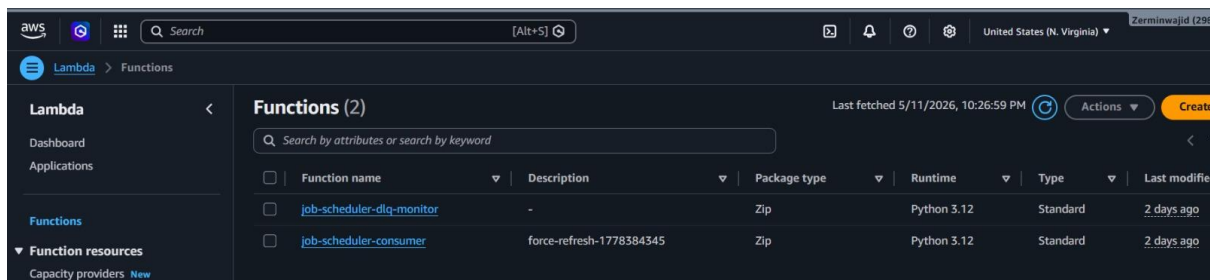
Objective	Implementation	Status
Serverless job ingestion pipeline using SQS and Lambda	SQS main queue + Consumer Lambda with event source mapping (batch=1, ReportBatchItemFailures)	IMPLEMENTED

Objective	Implementation	Status
Failure handling via Dead-Letter Queues and exponential backoff	maxReceiveCount=3 redrive policy;change_message_visibility backoff; DLQ Monitor Lambda	IMPLEMENTED
Persist job lifecycle state in Amazon RDS PostgreSQL	jobs table (current state) + job_events table (audit log); 5 status ENUM values	IMPLEMENTED
Live monitoring dashboard showing queue depth, failure rates, retry counts, DLQ contents	Flask dashboard with 4 JSON endpoints, Chart.js charts, KPI cards; polls every 3s	IMPLEMENTED
Demonstrate resilience through failure injection and load simulation	always_fail job type forces DLQ; loadtest command sends 50-200 concurrent jobs; failure_simulation.py runs 4 scenarios	IMPLEMENTED



Name	Type	Created	Messages available	Messages in flight	Encryption
job-scheduler-dlq	Standard	2026-05-10T04:41+05:00	0	0	Amazon SQS key (SSE-SQS)
job-scheduler-main	Standard	2026-05-10T04:57+05:00	0	0	Amazon SQS key (SSE-SQS)

Figure 1: SQS main queue



Function name	Description	Package type	Runtime	Type	Last modified
job-scheduler-dlq-monitor	-	Zip	Python 3.12	Standard	2 days ago
job-scheduler-consumer	force-refresh-1778384345	Zip	Python 3.12	Standard	2 days ago

Figure 2: Consumer Lambda

Dead-Letter Queue contents					Recent job events (audit log)			
Job ID	Type	Retries	Error	When	When	Status	Type	Job ID
fc9d6c15...	always_fail	3	RuntimeError: This job is designed to always fail	9:57:19 PM	9:59:26 PM	DLQ	flaky	a41ab48b...
6ac987c4...	flaky	3	RuntimeError: Simulated flaky failure	9:57:19 PM	9:59:20 PM	DLQ	always_fail	28631512...
28631512...	always_fail	3	RuntimeError: This job is designed to always fail	9:57:12 PM	9:59:12 PM	DLQ	always_fail	7cd1d8ab...
a41ab48b...	flaky	3	RuntimeError: Simulated flaky failure	9:57:10 PM	9:59:05 PM	DLQ	always_fail	fc9d6c15...
7da77ccd...	always_fail	3	RuntimeError: This job is designed to always fail	9:57:07 PM	9:59:02 PM	DLQ	always_fail	b28a722b...
b28a722b...	always_fail	3	RuntimeError: This job is designed to always fail	9:57:03 PM	9:59:00 PM	DLQ	flaky	6ac987c4...
7cd1d8ab...	always_fail	3	RuntimeError: This job is designed to always fail	9:56:59 PM	9:58:59 PM	DLQ	always_fail	7da77ccd...
e3c5c3d4...	echo	0		10:09:54 AM	9:57:19 PM	DLQ	always_fail	fc9d6c15...
61c980b0...	flaky	0		9:38:55 AM	9:57:19 PM	SUBMITTED	always_fail	fc9d6c15...
7f13cf12...	always_fail	0		9:38:55 AM	9:57:19 PM	STARTED	always_fail	fc9d6c15...
661e5238...	echo	0		9:38:55 AM	9:57:19 PM	DLQ	flaky	6ac987c4...
3b220266...	sleep	0		9:38:55 AM	9:57:19 PM	SUBMITTED	flaky	6ac987c4...
2ca7c8c6...	flaky	0		9:38:55 AM	9:57:19 PM	STARTED	flaky	6ac987c4...
e54732f4...	echo	0		9:38:55 AM	9:57:12 PM	DLQ	always_fail	28631512...
be153f69...	echo	0		9:38:55 AM	9:57:12 PM	SUBMITTED	always_fail	28631512...
f3858acc...	echo	0		9:38:55 AM	9:57:12 PM	STARTED	always_fail	28631512...
ba29f873...	flaky	0		9:38:55 AM	9:57:12 PM	STARTED	always_fail	28631512...
c49bc114...	flaky	0		9:38:55 AM	9:57:10 PM	DLQ	flaky	a41ab48b...

Figure 3: Dead Letter Queue (live demo)

The screenshot displays the AWS Management Console for an Amazon RDS database instance named 'jobscheduler-db'. The left sidebar shows the navigation menu with options like Dashboard, Databases, Performance insights, Snapshots, Exports in Amazon S3, Automated backups, Reserved instances, Proxies, Subnet groups, Parameter groups, Option groups, Custom engine versions, Zero-ETL integrations, Events, and Event subscriptions. The main content area shows the instance details and configuration options.

Summary

- DB identifier: jobscheduler-db
- Status: Available
- Class: db.t4g.micro
- Role: Instance
- Engine: PostgreSQL
- Region & AZ: us-east-1f
- Current activity: 0.00 sessions
- Recommendations: 2 Informational

Connectivity & security

- Connect using: Code snippets (selected), CloudShell, Endpoints
- Internet access gateway: Disabled
- Programming language: PSQL (macOS)
- Endpoint type: Instance endpoint
- IAM Authentication: Disabled

Figure 4: RDS creation

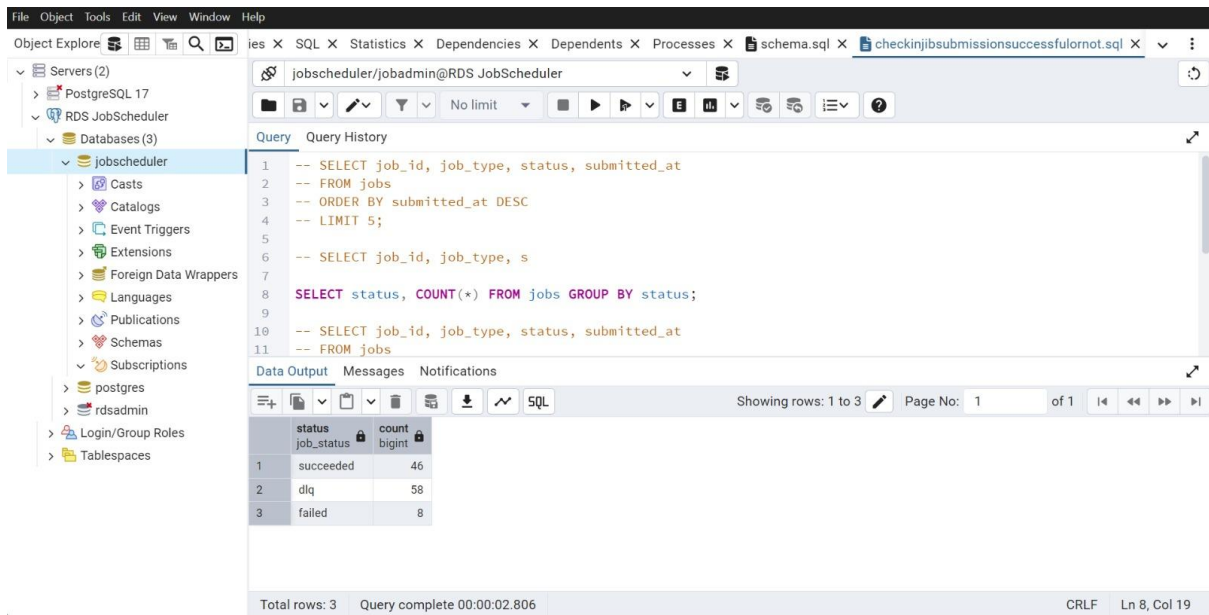


Figure 5: PostgreSQL

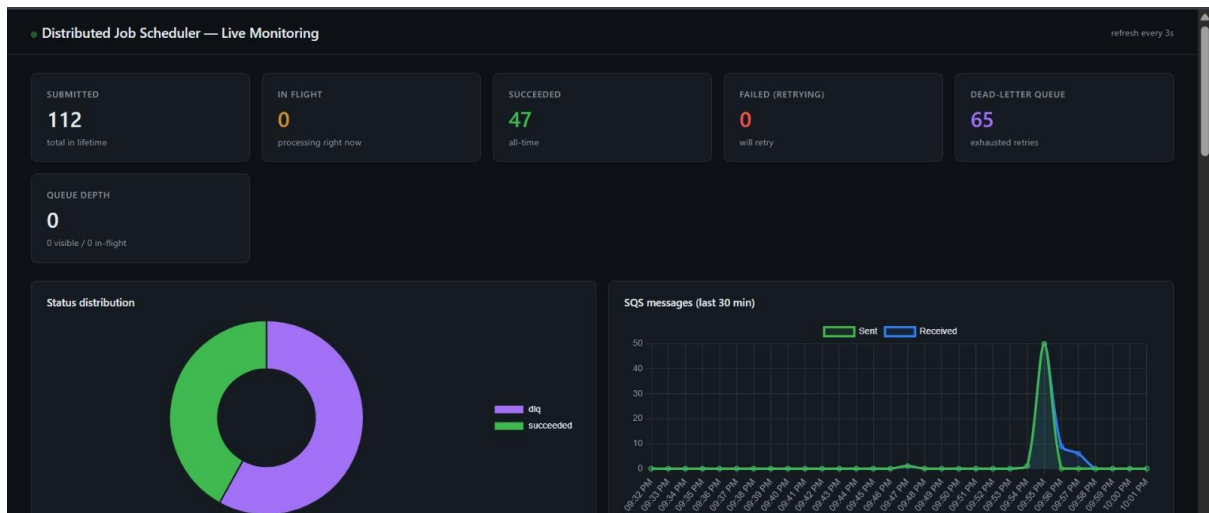


Figure 6: Live monitoring dashboard

AWS Infrastructure - Deployed Resources

SQS Queues

- Main Queue: job-scheduler-main
- VisibilityTimeout: 60 seconds
- MessageRetentionPeriod: 4 days
- RedrivePolicy: maxReceiveCount=3, deadLetterTargetArn=job-scheduler-dlq
- Dead-Letter Queue: job-scheduler-dlq (14-day retention)

Lambda Functions

- job-scheduler-consumer: Python 3.12, 256MB, 30s timeout; triggered by SQS main queue
- job-scheduler-dlq-monitor: Python 3.12, 256MB, 30s timeout; triggered by SQS DLQ

- Both deployed with Linux-compatible psycpg2-binary (manylinux2014_x86_64 wheel)
- Environment variables: DB_HOST, DB_NAME, DB_USER, DB_PASSWORD, QUEUE_URL, DLQ_URL, MAX_RECEIVES

RDS PostgreSQL

- Instance: db.t3.micro, PostgreSQL 15, Single-AZ
- Database: jobscheduler
- Tables: jobs (primary state), job_events (audit log)
- Views: v_status_counts, v_recent_dlq
- Public access enabled with security group restricting to 0.0.0.0/0 for Lambda connectivity

IAM Role

- Role: job-scheduler-lambda-role
- Policies: AWSLambdaBasicExecutionRole, AmazonSQSFullAccess, CloudWatchFullAccess

Deployment Steps

Step 1: RDS Database Creation

Created Amazon RDS PostgreSQL 15 instance (db.t3.micro, free tier) via AWS Console. Configured public access and security group inbound rules. Applied schema.sql via pgAdmin Query Tool to create the jobs and job_events tables.

Step 2: Infrastructure Provisioning

Ran infrastructure/setup_aws.py which created the SQS DLQ, main queue with redrive policy (maxReceiveCount=3), IAM Lambda execution role, and both Lambda functions. Event source mappings were created to wire SQS queues to their respective Lambda functions.

Step 3: Linux-Compatible Lambda Packaging

A critical deployment challenge was resolved during this phase. The initial Lambda deployment bundled a Windows-compiled psycpg2-binary, which crashed on Lambda's Linux runtime with os.add_dll_directory error. The fix was to download the correct Linux wheel explicitly:

```
pip install psycpg2-binary --platform manylinux2014_x86_64 --target build/consumer_linux --python-version 3.12 --only-binary=:all:
```

The Lambda code was then rezipped using Python's built-in zipfile module (zip command unavailable on Windows Git Bash) and redeployed. This produced a Linux-compatible deployment package that runs correctly on Lambda.

Step 4: Environment Variable Configuration

Updated both Lambda functions with correct DB_HOST, DB_PASSWORD, QUEUE_URL, DLQ_URL, and MAX_RECEIVES environment variables using the AWS CLI update-function-configuration command.

Step 5: Verification

Verified end-to-end pipeline by invoking Lambda directly with a base64-encoded test payload. Lambda successfully connected to RDS, inserted a job row, executed the echo job type, and returned `batchItemFailures: []` confirming success. pgAdmin confirmed the row appeared in the jobs table with `status = succeeded`.

Step 6: Dashboard

Launched the Flask dashboard locally (`python dashboard/app.py`). The dashboard connected to RDS and SQS, rendering live KPI cards and charts at `http://localhost:5000`.

Challenges Encountered and Solutions

Challenge	Root Cause	Solution
<code>os.add_dll_directory</code> error on Lambda	psycpg2-binary installed on Windows was bundled into the zip; Lambda runs Linux	Installed Linux wheel explicitly using <code>--platform manylinux2014_x86_64</code>
Bash script JSON quoting failures	Git Bash on Windows mangles JSON strings inside shell variables	Replaced entire <code>setup_aws.sh</code> with <code>setup_aws.py</code> using Python boto3 (no quoting issues)
pgAdmin SSL connection refused	RDS requires SSL by default; pgAdmin was sending unencrypted connections	Set SSL mode = require in pgAdmin Parameters tab
Lambda never triggered by SQS	<code>DB_HOST</code> was still <code>FILL_THIS_IN</code> placeholder; Lambda crashed silently before logging	Updated env vars via <code>aws lambda update-function-configuration</code>
<code>zip</code> command not found in Git Bash	<code>zip</code> utility not installed on Windows by default	Used Python <code>zipfile</code> module to create deployment packages
<code>MSYS_NO_PATHCONV</code> paths mangled by Git Bash	Git Bash converts <code>/aws/lambda/</code> paths to Windows paths	Prefixed affected commands with <code>MSYS_NO_PATHCONV=1</code>

Conclusion

The Distributed Job Scheduling and Monitoring System was successfully designed, deployed, and demonstrated on live AWS infrastructure within the three-week project timeline. All five objectives stated in the original proposal were implemented and verified.

The project demonstrates direct application of CE308 core concepts: serverless compute (Lambda), managed queuing (SQS), relational persistence (RDS), observability (CloudWatch + custom dashboard), and fault-tolerant architecture (DLQ + retry policies). The system goes beyond prior assignments in integrating five AWS services under a unified architecture with real failure-handling semantics, stateful job tracking, and operational observability.

Key technical achievements include the correct implementation of exponential backoff via SQS visibility timeout extension, the split jobs/job_events database design for efficient state queries alongside a complete audit trail, partial-batch failure responses to avoid penalizing healthy messages in a failed batch, and the resolution of a non-trivial platform compatibility challenge (Windows-to-Linux psycpg2 deployment).

The live dashboard, accessible at <http://localhost:5000> during the demo, provides real-time visibility into all aspects of the system and directly satisfies the operational observability requirement of the proposal.